



Universidad autónoma de baja california

Ingeniería en computación

Internet de las cosas

Lab-taller 7: SMS

Aguilar Noriega Leocundo

Garcia Chaves erik

martes 10 de marzo del 2026

Instrucciones:

Realizar un prototipo de IoT basado en ESP32 que pueda acceder a una base de datos remota y pues escribir datos relacionadas a sus recursos o atributos

Desarrollo:

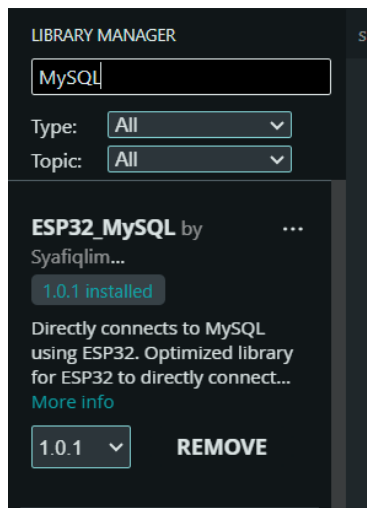
Parte 1:

Haciendo uso de la plataforma **Arduino** realizar un programa capaz de acceder a la base de datos MySQL

Mediante las credenciales dadas en la practica acceder y cargar datos en la tabla llamada **data** la cual tiene las siguientes columnas:

- **Device:** corresponde a la identificacion del dispositivo, para casos practicos se utilizara la matricula del alumno
- **RSSI:** es la referencia para medir el nivel de potencia de las senales recibidas por un dispositivo en las redes inalambricas, un valor que ronda el rango de 0 a -80, siendo el 0 el valor de la senal mas fuerte
- **IP:** corresponde a la direccion IP publica que actualmente esta usando la infraestructura de conectividad a internet con la cual se esta conectando el dispositivo ESP32
- **LED:** el estado actual de un led que puede ser 1 (**ON**) o 0 (**OFF**)
- **ADC:** valor de lectura del ADC.

Para el desarrollo de la practica se utilizo la libreria **ESP32_MySQL**



Esta librería ya contiene módulos que permiten la conexión y la inserción por medio de queries a la base de datos.

Para poder obtener los datos que necesitamos en la tabla, es necesario el uso de variables globales, arreglos de caracteres que indicaran en la QUERY el nombre de la columna que va a insertar la información

```
//columnas de la tabla
//nombre de las columnas dentro de la tabla
char device_column[]="DEVICE";
char RSSI_column[] = "RSSI";
char IP_column[] = "IP";
char LED_column[] = "LED";
char ADC_column[] = "ADC";
```

Las siguientes variables globales se encargan de recopilar la información que tiende a variar como el rssi, el estado del led, y del ADC, la IP esa solo se necesita obtener 1 vez y la obtenemos dentro de la función **setup()**,

```
//recopilar datos
char IP_PUBLIC[16];
//guardar el RSSI de la serial wifi
long rssi;
char device_value[]="1275863";

//LED / ADC

int LED_VALUE = 0;
int ADC_VALUE = 0;
```

Para obtener la dirección IP se realiza mediante la siguiente función, realizando una petición a la API de api.ipify.org

```
void getIP_Public(){
    if(WiFi.status()== WL_CONNECTED){
        HTTPClient http;
        http.begin("http://api.ipify.org"); // Servicio para obtener IP
        int httpCode = http.GET();
        if (httpCode > 0) {
            String payload = http.getString();
            Serial.println("IP Pública: " + payload);
            strncpy(IP_PUBLIC, payload.c_str(), 15);
            IP_PUBLIC[15] = '\0'; // asegurar terminación null
        }
        http.end();
    }
}
```

En la función **setup** como se mencionó se inician pines, y se muestra hacia de base de datos se realizará la inserción de los datos

```
void setup() {
    // put your setup code here, to run once:
    Serial.begin(115200);

    //inicia WIFI
    initWiFi();

    pinMode(pinLed, OUTPUT); // -> este es para el PIN
    pinMode(pinButton, INPUT); // -> PIN de entrada del boton

    //obtenemos la IP publica del internet
    getIP_Public();

    ESP32_MYSQL_DISPLAY3("Connecting to SQL Server @", server, ", Port =", server_port);
    ESP32_MYSQL_DISPLAY5("User =", user, ", PW =", password_DB, ", DB =", database);
}
```

La función **runInsert()** es la función encargada de estructurar la query que realizara la inserción a la base de datos, este que construye con la ayuda del tipo de dato String de Arduino y los módulos de la libreria

```
void runInsert()
{
    // Initiate the query class instance
    ESP32_MySQL_Query query_mem = ESP32_MySQL_Query(&conn);

    if (conn.connected())
    {
        ESP32_MYSQL_DISPLAY("Database connected. Inserting data...");
        String INSERT_SQL = String("INSERT INTO ") + database + "." + table
            + " (" + device_column + "," + RSSI_column + "," + IP_column + "," + LED_column + "," + ADC_column + ") VALUES ("
            + device_value + "," + rssi + "," + IP_PUBLIC + "," + LED_VALUE + "," + ADC_VALUE + ")";
        ESP32_MYSQL_DISPLAY(INSERT_SQL);

        // Execute the query
        if ( !query_mem.execute(INSERT_SQL.c_str()) )
        {
            ESP32_MYSQL_DISPLAY("Insert error");
        }
        else
        {
            ESP32_MYSQL_DISPLAY("Data Inserted.");
        }
    }
    else
    {
        ESP32_MYSQL_DISPLAY("Error connecting to Database. Can't insert.");
    }
}
```

En la función **loop** vamos capturando los datos necesarios que tienden a variar, como lo es el RSSI, el estado del led y del ADC. Primero se tiene la opción de cambiar el estado del led, mediante toggle esto para que el proceso de cambiar de estado no se bloquee durante el periodo que debe de esperar para volver a insertar un nuevo dato a la base de datos.

Cuando el tiempo, los 5 minutos en este caso hayan pasado el proceso de insertar datos va a proceder, obtenemos los valores que tienden a cambiar, y mediante las funciones de la libreria se trata de realizar la conexión a la base de datos, si es correcta la conexión entonces se manda a llamar a la función que contiene la query, en otro caso se indica que ha fallado. Y este procedimiento se estaría realizando cada 5 minutos.

```

void loop() {
    bool currentButtonState = digitalRead(pinButton);

    if (currentButtonState == HIGH && lastButtonState == LOW) {
        ledState = !ledState;
        digitalWrite(pinLed, ledState);
        Serial.println(ledState ? "LED ON" : "LED OFF");
        delay(50);
    }

    lastButtonState = currentButtonState;
    unsigned long now = millis();

    if (now - lastSendTime >= SEND_INTERVAL) {
        lastSendTime = now;

        rssi = WiFi.RSSI();
        LED_VALUE = ledState ? 1 : 0;
        ADC_VALUE = analogRead(pin_ADC);

        ESP32_MYSQL_DISPLAY("comunicacion con la DB...");
        if (conn.connectNonBlocking(server, server_port, user, password_DB) != RESULT_FAIL)
        {
            delay(50);
            runInsert();
            conn.close(); // close the connection
        }
        else
        {
            ESP32_MYSQL_DISPLAY("\nConnect failed. Trying again on next iteration.");
        }
    }

    ESP32_MYSQL_DISPLAY("\nSleeping...");
}

```

```

    ESP32_MYSQL_DISPLAY("\nSleeping...");
    ESP32_MYSQL_DISPLAY("=====");
}
}

```

Ejecución del programa:

2026-03-10 05:20:56	20240001	187.189.207.11	-62	0	0
2026-03-10 05:21:20	273892	0.0.0.0	-58	0	0
2026-03-10 05:29:44	273892	187.189.207.11	-55	0	4095
2026-03-10 15:17:36	273892	200.68.171.21	-52	0	4095
2026-03-10 15:20:21	273892	200.68.171.21	-53	0	4095
2026-03-10 15:21:37	273892	200.68.171.21	-58	0	765
2026-03-10 17:02:37	1275863	187.250.234.157	-59	1	2261
2026-03-10 17:03:36	1275863	187.250.234.157	-65	1	2257
2026-03-10 17:04:36	1275863	187.250.234.157	-61	0	2258

```

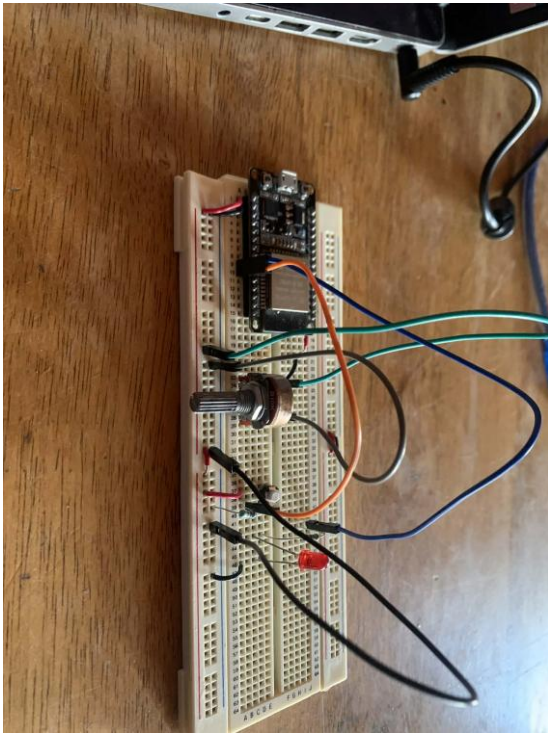
Sleeping...
=====
LED OFF
comunicacion con la DB...
Database connected. Inserting data...
INSERT INTO u598733893_1ot_db.data (DEVICE,RSSI,IP,LED,ADC) VALUES ('1275863', -61,'187.250.234.157',0,225);
Data Inserted.
Sleeping...
=====

```

Ln 45, Col 37 ESP32 Dev Module on COM3

Podemos ver que la inserción de los datos los datos de manera correcta en la tabla.

Hardware:



Conclusiones:

Durante el desarrollo de la práctica, se pudo realizar la inserción a una base de datos lo cual es algo crucial, lo cual le dará sentido a las acciones que un sistema IoT está tomando, el registrar, guardar lo que está pasando con su entorno puede ser herramientas para un posible análisis e indicar tendencias o tal vez como un dispositivo que está registrando el clima en un campo de siembra para poder pronosticar sequias, posibles lluvias, etc. Porque hay un registro histórico y eso solo se podría si hay una base de datos en donde se está enviando la información, es uno de los topics importantes dentro del IoT, después podría ser la interconectividad de distintos dispositivos de IoT.

Por lo que lo único que se tuvo dificultades es el construir la query, dado de C++ no es un lenguaje que nativamente tiene esa posibilidad, pero con la ayuda de librerías se logro realizar.

Referencias:

[ESP32 Useful Wi-Fi Library Functions \(Arduino IDE\) | Random Nerd Tutorials](#)

[Syafiqlim/ESP32 MySQL: Optimized library for ESP32 to directly connect and execute SQL to MySQL database from ESP32 without intermediary](#)

[How to read the state of an output pin ? - Projects / Programming - Arduino Forum](#)

[Arduino - Turn LED ON and OFF With Button - The Robotics Back-End](#)

[¿Cómo leer el estado de un pin de salida? - Programación - Foro Arduino](#)